LibRec: Comparing Social-Aware Recommender Systems

Cale Smith, Advait Shinde, Jasper Morgal
Department of Computer Engineering
San Jose State University
San Jose, CA, USA
{cale.smith, advait.shinde, jasper.morgal}@sjsu.edu

***Abstract*—Recommender systems have a tendency to struggle with sparse ratings, especially in cold-start settings. In this project we attempt to beat simple baselines by leveraging trust graphs. To do this we use the LibraryThing dataset (1.7M reviews and 83K users) which contains the classic (user, item, rating) tuple as well as a trust graph. We build a shared pipeline along with two baselines, a global mean and an untuned LightGBM model, and then attempt to beat those baselines across rating-accuracy and ranking-quality metrics. To do this we implement three models: a hyper-tuned LightGBM model, a Neural Collaborative Filtering (NCF) model with social regularization, and Graph Feature Propagation with Gradient Boosting (GFP-GB).**

Repository: https://www.github.com/calepayson/librec

I. Introduction

Collaborative filtering (CF) is one of the fundamental strategies of recommender systems but, despite improvements, still struggles with sparse user-item interaction matrices. Our chosen dataset, LibraryThing, is incredibly sparse, with only 0.0041% of user-item combinations being filled in. Compounding the difficulty, only 38% of users have rated more than five items.

While this would make standard collaborative filtering approaches quite unreliable, LibraryThing also includes other metadata, the most important addition being a friendship graph (with 219K edges). This social signal gives us the opportunity to test social-aware recommender systems against classic baselines. With that in mind our goal is to see how various strategies perform against the baseline.

We test this with a comparative study. We create a shared pipeline that ensures each model shares the same preprocessing step and is evaluated on the same metrics. The models are as follows:

- **Variant A**: (Baseline): Global Mean - Simply returns the average rating of the training set.
- **Variant B**: (Baseline): Untuned-LightGBM - A LightGBM model with default hyperparameters.
- **Variant C** (Jasper): Hyper-tuned LightGBM - Takes the baseline and evaluates the performance difference of aggressive feature engineering, and hyperparameter tuning.
- **Variant D** (Cale): SocialNCF - Combines neural collaborative filtering (NCF) with social graph regularization and neighbor-averaging cold-start inference.
- **Variant E** (Advait): GFP-GB — Graph Feature Propagation with Gradient Boosting. Propagates handcrafted statistical features through the bipartite user-item graph and social graph, then trains LightGBM on the resulting feature matrix.

II. Dataset Analysis

We use the LibraryThing dataset from the McAuley Lab collection [1], which pairs user–item ratings with a social friendship graph. Table I summarizes its key statistics.

TABLE I
LibraryThing Dataset Statistics

Metric	Value
Reviews	1,707,070
Users	83,195
Items	506,165
Social edges	219,790
Sparsity (%)	0.0041
Users $\leq$5 ratings (%)	62.0
Users in social graph (%)	30.8
Cold users w/ social (%)	21.5
User Gini	0.809
Item Gini	0.632
Mean rating	3.81
Rating std	1.00

LibraryThing is a book review platform with 1.7M ratings from 83K users across 506K works. Ratings use a half-star scale (0.5-5.0) and peak at 4.0 stars (Figure 1), suggesting that users tend to rate books they enjoyed. The user activity distribution is highly unequal (Gini = 0.809), a small core of power users contributes disproportionately, while 62% of users have five or fewer ratings (Figure 2). The social friendship graph covers 30.8% of users, and among cold-start users ($\leq$5 ratings), 21.5% have at least one social connection which provides a meaningful signal for social-aware methods.

III. Shared Pipeline and Baselines

Our pipeline uses a modular structure where each module produces a cached artifact that following stages can use. This makes it easy to collaborate on the project without having having to rebuild the entire pipeline on every change.

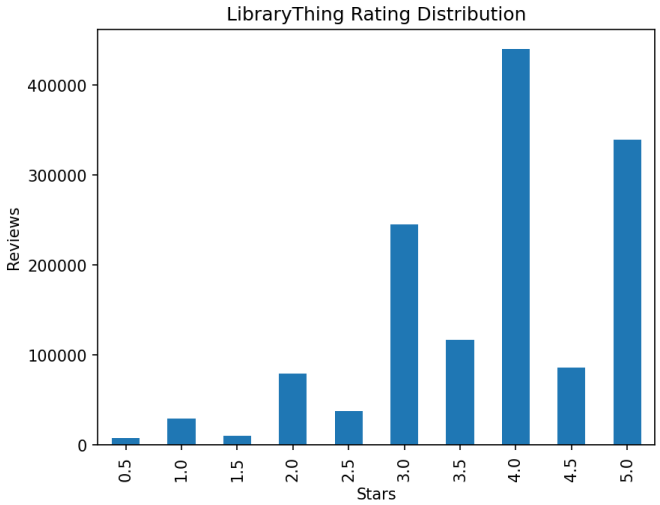

Fig. 1. LibraryThing rating distribution.

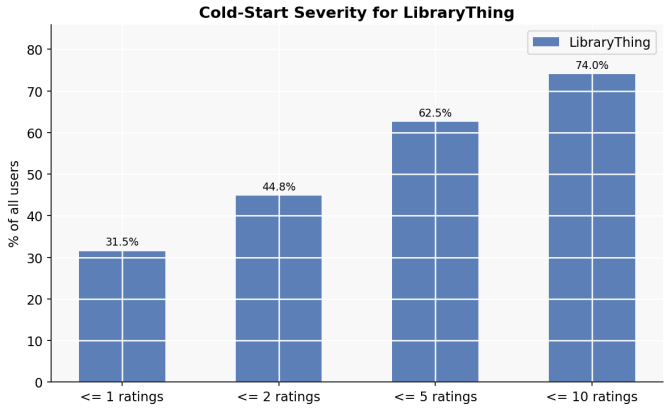

Fig. 2. LibraryThing rating distribution.

A. Preprocessing

We process the raw data in a few steps, each of which produces a cached artifact. First we convert the raw review data into the standard (user, item, rating) format, although we also include a time feature. We then use the time feature to apply temporal 80/10/10 split, where the train set has the oldest 80% of data, the val set has the next oldest 10%, etc.

After splitting the data we transform it. First, user and item identifiers are encoded as integers using the training set identifiers. Users or items that don't appear in the training set (cold-start) are given a value of -1 though we also extend the codes for socially aware models. Second, we normalize the user ratings per user. Third, we update the temporal feature to reflect time since the first datapoint rather than unix time.

We also update the LibraryThing friendship graph to reflect the new user encodings, discarding any users who aren't included in the training data.

B. Evaluation Framework

All models are evaluated using the same framework implemented in a shared BaseModel class. This BaseModel presents a shared interface to be used by each model and computes two classes of metrics:

Rating accuracy: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), computed on both validation and test sets. These measure how well a model predicts the exact star rating a user would assign.

Ranking quality: Precision@10, Recall@10, NDCG@10, and Hit Rate@10. For these, we define a rating ≥ 4.0 as "relevant." For each user, we take the top 10 items by predicted score and compute precision (fraction of top-10 that are relevant), recall (fraction of all relevant items captured in top-10), NDCG (position-weighted relevance), and hit rate (whether at least one relevant item appears in top-10).

C. Baseline: Global Mean

Our simplest baseline predicts the training-set mean rating (3.81) for every user-item pair. This model is trivial but establishes a floor: any method that cannot beat global mean is not learning useful patterns from the data.

D. Baseline: LightGBM

Our second baseline is a LightGBM [2] gradient-boosted decision tree regressor. It takes a user code and an item code as categorical input features and predicts the rating. Training uses 500 boosting rounds with early stopping on validation RMSE. Key hyperparameters include 63 leaves per tree, learning rate 0.05, and 90% feature/bagging fractions. Unseen users and items in the validation and test sets receive code -1, which LightGBM treats as a distinct category. This baseline is substantially stronger than global mean, as it can learn per-user and per-item biases.

IV. Hyper-Tuned LightGBM

Jasper Morgal

A. Design and Motivation

The hyper-tuned LightGBM model was built to push the limits of simple gradient boosting, which is known to work well in cold start environments. The idea was to augment the strong baseline of LightGBM with tons of engineered features from the social interaction graph. Lots of aggregate features were also added like the user's average rating, or rating count. Out-of-fold feature construction was also an important part of this model, as it helped prevent overfitting during training.

B. Implementation Details

The LightGBM model is tasked with predicting user-item rating pairs given 62 unique features. It first splits the data into 5 folds and computes features from data outside of each fold. This prevents each row's data from contaminating features like the mean when there are a low number of ratings for that feature.

The model also loads the social trust graph, and computes features from the trusted neighbors, like average rating for

example. These features are also computed out of fold and are intended to help the model predict when user-item history alone is insufficient.

Unfortunately, the model tends to favor only a few top features like `user_item_bias_sum` and `mean_user_item_rating`, however any attempt to prioritize features more evenly strictly decreased performance. Also, the social trust graph features were hardly used at all by the model.

See Table II for a list of every feature included in the model.

C. Design Decision Log

See Table III.

D. Results and Analysis

The hyper-tuned LightGBM achieved validation RMSE 0.9480 and test RMSE 0.9632, improving over the untuned LightGBM baseline's 0.9551 validation RMSE and 0.9690 test RMSE. Test MAE also improved from 0.7510 to 0.7434.

These results show a small but consistent increase from the LightGBM baseline, as well as representing the highest scores our team achieved on this task. While I am somewhat disappointed in the high concentration of feature importance of the top 2 features, I am overall satisfied with the model's high performance.

V. SocialNCF

Cale Smith

A. Design and Motivation

The LightGBM baselines treat user and item codes as categorical features, learning per-entity biases but not dense interactions. Neural Collaborative Filtering (NCF) [3] addresses this by learning dense embedding vectors for each user and item and combining them through both linear (GMF) and nonlinear (MLP) pathways. Also, I like deep learning and just wanted to add a neural net.

However, NCF learns embeddings just from rating interactions. Since our exploratory data analysis shows that 21.5% of cold-start LibraryThing users have social connections, I wondered if the rating could be improved with that info. Social-Regularized NCF extends NCF by incorporating the social graph in two ways:

1) **Training-time regularization:** A penalty term encourages the GMF embeddings of socially connected users to be similar. I based this off of Ma et al. [4].
2) **Cold-start inference:** For users with no training ratings but at least one social connection, I used the mean of their freinds' embeddings (both GMF and MLP).

B. Implementation Details

I implemented the NeuMF architecture based on He et al. [3]. The model has four embeddings: user and item embeddings for the GMF path, and user and item embeddings for the MLP path, each with 32 dimensions (kept it at 32 because my computer is a potato). The GMF path computes element-wise products of user and item embeddings. The MLP path concatenates user and item embeddings (64-dimensional input) and passes them through two hidden layers of size 64 and 32 with ReLU activations and 10% dropout. The GMF output and MLP output are concatenated and fed to a linear prediction layer that produces the final rating.

I used the Adam optimizer with a learning rate of 10^{-3}, a batch size of 8192, and MSE loss. I used early stopping after 5 epochs of no improvement on the validation set. The linear layer weights were initialized with Xavier uniform distribution and normal distribution for embeddings.

The social regularization term is added to the loss at each training step:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \cdot \frac{1}{|E|} \sum_{(u,v)\in E} \|\mathbf{p}_u - \mathbf{p}_v\|^2 \tag{1}$$

where E is the set of trust/friendship edges between training-set users, $\mathbf{p}_u$ denotes the GMF embedding of user u, and $\lambda = 0.01$.

To handle cold-start users, the preprocessing pipeline assigns integer codes to all users: training users receive codes 0 to $n_{\text{train}} - 1$, while users in only validation/test receive codes n_{train} and above. At inference time, a cold-start user u with neighbor set $\mathcal{N}(u)$, is computed with:

$$\hat{\mathbf{p}}_u = \frac{1}{|\mathcal{N}(u)|} \sum_{v\in\mathcal{N}(u)} \mathbf{p}_v \tag{2}$$

and same for the MLP embedding. These averaged embeddings are passed through the full NeuMF forward pass to produce predictions. Unseen users with no social connections are just assigned the global average rating as a prediction.

C. Design Decisions

D. Results and Analysis

The base NCF model (without social regularization) scored a test RMSE of 0.9922 and MAE of 0.7811, slightly worse than the untuned LightGBM baseline (RMSE 0.9690). Adding social regularization brought the test RMSE down marginally to 0.9919 and improved MAE to 0.7794. More notably, Social NCF improved ranking metrics over base NCF: Precision@10 rose from 0.3045 to 0.3074 and NDCG@10 from 0.7379 to 0.7422. This suggests the social regularization term helps the model learn embeddings that better capture user preference, even if the rating accuracy improvement is small.

The cold-start neighbor-averaging strategy allows Social NCF to produce personalized predictions for users who have no training ratings but do have friends in the graph. Without this, those users would just receive the global mean prediction.

But, all that being said, it didn't beat the simple LightGBM baseline, despite many hours of hyperparameter tuning that I wish I could take back. Stick with the simple model first kids. Complexity is tough.

VI. GFP-GB: Graph Feature Propagation with Gradient Boosting

Advait Shinde

TABLE II
HYPER-TUNED LIGHTGBM — FEATURE IMPORTANCE

Feature	Type	Description	Importance
user_item_bias_sum	Interaction	Sum of user and item bias from the global mean.	52.01%
mean_user_item_rating	Interaction	Average of the user's mean rating and the item's mean rating.	30.16%
user_mean	User aggregate	Mean rating given by the user in training data.	5.28%
user_mean_minus_global	User aggregate	User mean rating minus the global training mean.	2.37%
item_count	Item aggregate	Number of training ratings for the item.	2.02%
user_raw_mean	User aggregate	Copy of the user's mean rating before derived interactions.	1.74%
user_count	User aggregate	Number of training ratings by the user.	1.21%
user_bias	User aggregate	User rating bias relative to the global mean.	0.91%
item_min	Item aggregate	Minimum training rating received by the item.	0.83%
user_item_bias_gap	Interaction	Difference between user bias and item bias.	0.68%
user_median	User aggregate	Median rating given by the user.	0.59%
item_log_count	Item aggregate	Log-transformed item rating count.	0.39%
user_log_count	User aggregate	Log-transformed user rating count.	0.38%
item_max	Item aggregate	Maximum training rating received by the item.	0.36%
item_raw_mean	Item aggregate	Copy of the item's mean rating before derived interactions.	0.20%
social_item_user_gap	Social item feature	User mean minus trusted-neighbor mean rating for the target item.	0.14%
user_item_popularity	Interaction	Product of log user count and log item count.	0.14%
item_mean	Item aggregate	Mean training rating received by the item.	0.12%
social_item_item_gap	Social item feature	Item mean minus trusted-neighbor mean rating for the target item.	0.11%
item_mean_minus_global	Item aggregate	Item mean rating minus the global training mean.	0.11%
item_minus_user_mean	Interaction	Item mean rating minus user mean rating.	0.08%
user_max	User aggregate	Maximum rating given by the user.	0.07%
item_rating_span	Item aggregate	Difference between the item's maximum and minimum rating.	0.04%
item_median	Item aggregate	Median training rating received by the item.	0.03%
is_cold_item	Cold-start flag	Indicates whether the item is unseen in training aggregates.	0.01%
item_std	Item aggregate	Standard deviation of ratings for the item.	0.01%
user_std	User aggregate	Standard deviation of the user's ratings.	0.00%
user_min	User aggregate	Minimum rating given by the user.	0.00%
user_item_train_count	User-item aggregate	Number of times this user-item pair appears in training.	0.00%
social_degree	Social aggregate	Number of trusted neighbors for the user.	0.00%
social_neighbor_count	Social aggregate	Number of trusted neighbors with training ratings.	0.00%
social_neighbor_rating_count	Social aggregate	Total number of ratings from trusted neighbors.	0.00%
social_neighbor_mean	Social aggregate	Average rating mean across trusted neighbors.	0.00%
social_neighbor_mean_std	Social aggregate	Standard deviation of trusted-neighbor mean ratings.	0.00%
social_neighbor_mean_min	Social aggregate	Minimum mean rating among trusted neighbors.	0.00%
social_neighbor_mean_max	Social aggregate	Maximum mean rating among trusted neighbors.	0.00%
social_neighbor_weighted_mean	Social aggregate	Trusted-neighbor mean weighted by neighbor rating count.	0.00%
social_item_neighbor_count	Social item feature	Number of trusted neighbors who rated the target item.	0.00%
social_item_neighbor_mean	Social item feature	Mean rating trusted neighbors gave to the target item.	0.00%
social_item_neighbor_std	Social item feature	Standard deviation of trusted-neighbor ratings for the target item.	0.00%
social_item_neighbor_min	Social item feature	Minimum trusted-neighbor rating for the target item.	0.00%
social_item_neighbor_max	Social item feature	Maximum trusted-neighbor rating for the target item.	0.00%
social_item_neighbor_frac_liked	Social item feature	Fraction of trusted-neighbor item ratings at least 4 stars.	0.00%
social_item_neighbor_frac_disliked	Social item feature	Fraction of trusted-neighbor item ratings at most 2 stars.	0.00%
is_cold_user	Cold-start flag	Indicates whether the user is unseen in training aggregates.	0.00%
user_rating_span	User aggregate	Difference between the user's maximum and minimum rating.	0.00%
item_bias	Item aggregate	Item rating bias relative to the global mean.	0.00%
social_log_degree	Social aggregate	Log-transformed social degree.	0.00%
social_neighbor_log_rating_count	Social aggregate	Log-transformed trusted-neighbor rating count.	0.00%
social_neighbor_coverage	Social aggregate	Fraction of trusted neighbors with rating history.	0.00%
social_has_neighbors	Social flag	Indicates whether the user has trusted neighbors.	0.00%
social_has_rated_neighbors	Social flag	Indicates whether trusted neighbors have ratings.	0.00%
social_neighbor_mean_range	Social aggregate	Range of trusted-neighbor mean ratings.	0.00%
social_neighbor_mean_minus_global	Social aggregate	Trusted-neighbor mean minus the global training mean.	0.00%
social_neighbor_weighted_mean_minus_global	Social aggregate	Weighted trusted-neighbor mean minus the global training mean.	0.00%
social_user_vs_neighbor_mean	Social aggregate	User mean minus trusted-neighbor weighted mean.	0.00%
social_item_vs_neighbor_mean	Social aggregate	Item mean minus trusted-neighbor weighted mean.	0.00%
social_item_neighbor_log_count	Social item feature	Log-transformed count of trusted neighbors who rated the item.	0.00%
social_item_has_neighbor_rating	Social item flag	Indicates whether any trusted neighbor rated the target item.	0.00%
social_item_neighbor_share	Social item feature	Share of trusted neighbors who rated the target item.	0.00%
social_item_neighbor_range	Social item feature	Range of trusted-neighbor ratings for the target item.	0.00%
social_item_neighbor_mean_minus_global	Social item feature	Trusted-neighbor item mean minus the global training mean.	0.00%

TABLE III
HYPER-TUNED LIGHTGBM — KEY DESIGN DECISIONS

Choice	Selected	Alternatives Tried
Base model	LightGBM	Linear; neural
Features	Rating + social aggregates	Raw IDs
Aggregation	5-fold OOF	Full-train aggregates
Social signal	Trust-neighbor stats	No trust graph
Tuning	Validation RMSE sweep	Defaults
Tree size	15 leaves	31 leaves
Subsampling	Row + feature	None
Stopping	20-round early stop	Full 500 rounds

TABLE IV
SOCIALNCF — KEY DESIGN DECISIONS

Choice	Selected	Alternatives Tried
Architecture	SocialNeuMF	NeuMF
Embedding dim	32	64, 128
Learning rate	10^{-3}	5×10^{-4}, 10^{-4}
Loss function	MSE	RMSE
Reg. target	GMF embeddings	None
λ	0.01	0.001, 0.1, 1.0
Cold-start strategy	Social Mean	None

A. Design and Motivation

The LightGBM baseline learns per-entity biases from user and item codes, but ignores the graph structure entirely. End-to-end graph learning (e.g., LightGCN [5]) is an alternative, but on a dataset this sparse it tends to overfit quickly. GFP-GB takes a middle path: compute robust, interpretable statistical features from training interactions, propagate them through the graph to give each node neighborhood context, and then let LightGBM learn how to combine those features into a rating prediction.

The key advantage over learned embeddings is that propagated statistics are computed once from training data, require no gradient flow through the graph, and naturally handle cold-start nodes via shrinkage toward the global mean. Since Jasper's section already covers LightGBM tuning in depth, this section focuses on the graph feature propagation that distinguishes GFP-GB.

B. Implementation Details

Feature construction follows six steps, illustrated in Figure 3:

Step 1 — Node statistics. For every user and item in the training set, compute mean, count, and standard deviation of ratings. Raw means are shrunk toward the global mean with a shrinkage constant $K = 10$:

$$\tilde{\mu}_u = \frac{n_u \bar{r}_u + K\bar{r}}{n_u + K} \tag{3}$$

where n_u is the user's rating count and $\bar{r}$ is the global mean. The same formula applies to items.

Step 2 — Bipartite propagation (1-hop). Each item inherits the average shrunk mean of its raters; each user inherits the average shrunk mean of the items they rated:

$$\text{sm_item}_i = \frac{1}{|\mathcal{U}_i|} \sum_{u \in \mathcal{U}_i} \tilde{\mu}_u \tag{4}$$

$$\text{sm_user}_u = \frac{1}{|\mathcal{I}_u|} \sum_{i \in \mathcal{I}_u} \tilde{\mu}_i \tag{5}$$

Step 3 — Bipartite propagation (2-hop). Apply the same averaging one step further, so items inherit 2-hop user context and vice versa.

Step 4 — Social propagation (1-hop). Each user inherits the average shrunk mean of their social neighbors, giving a `neighbor_mean` feature and a `social_degree` feature.

Step 5 — Feature assembly. For each (user, item) pair, concatenate the original statistics, both bipartite-propagated features (1- and 2-hop), difference features (`item_vs_smoothed`, `user_vs_smoothed`), and social features into a 12-dimensional feature vector.

Step 6 — LightGBM training. A LightGBM regressor is trained on the assembled feature matrix with the same hyperparameters as the baseline: learning rate 0.05, 63 leaves, 500 rounds with early stopping on validation RMSE.

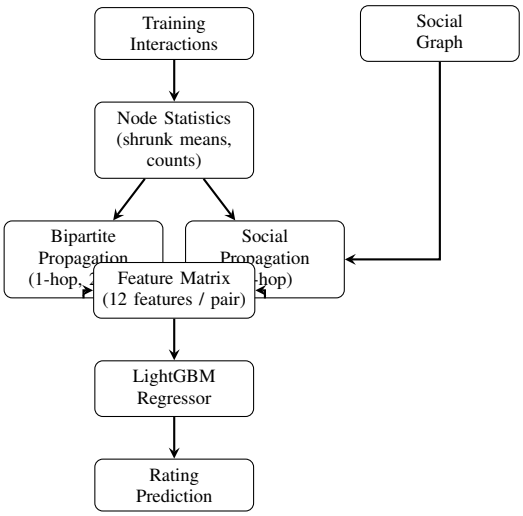

Fig. 3. GFP-GB architecture. Statistical features are propagated through both the bipartite interaction graph and the social graph before being fed to LightGBM.

C. Design Decision Log

D. Results and Analysis

GFP-GB achieves a test RMSE of 0.9708 and MAE of 0.7487, beating Global Mean on both accuracy metrics. The MAE (0.7487) is slightly better than the untuned LightGBM baseline (0.7510), but RMSE is marginally worse (0.9708 vs. 0.9690), suggesting that the propagated features help reduce typical errors but do not eliminate large outlier errors.

On ranking metrics, GFP-GB trails both LightGBM variants (NDCG@10 0.7209 vs. 0.7491), which is not surprising:

TABLE V
GFP-GB — KEY DESIGN DECISIONS

Choice	Selected	Alternatives Tried
Graph method	Statistical propagation	LightGCN [5] (overfit, val RMSE 0.9458 at epoch 4)
Predictor	LightGBM	GFP-MLP (val RMSE 0.9524 in notebook)
Prop. depth	2-hop bipartite, 1-hop social	1-hop only
Cold-start	Shrinkage to global mean	Code $= -1$
Feature type	Handcrafted statistics	Learned embeddings

LightGBM's per-entity codes already capture most of the user/item bias that drives ranking, and on a dataset this sparse the propagated neighborhood means add limited additional signal. The 2-hop bipartite and social features help the model generalize to cold-start and lightly-rated users, but the graph coverage (30.8% of users in the social graph) limits how far this benefit reaches.

VII. COMPARATIVE ANALYSIS AND DISCUSSION

A. Rating Prediction

On RMSE, the two LightGBM variants dominate: the untuned baseline (0.9690) already pushes well below Global Mean (0.9945). GFP-GB (0.9708) is competitive but does not beat it, despite adding 10 graph-derived features on top of the same user/item codes. Social NCF (0.9919) trails further behind, consistent with neural methods needing denser interaction data to outperform tree models. On MAE, GFP-GB (0.7487) narrowly edges the LightGBM baseline (0.7510), suggesting the propagated features reduce typical errors even when they do not reduce the largest ones.

B. Ranking Quality

LightGBM leads on all ranking metrics (NDCG@10 0.7491, P@10 0.3115). Adding graph features (GFP-GB, NDCG@10 0.7209) or social regularization (Social NCF, NDCG@10 0.7422) does not close this gap. The result suggests that per-entity bias captured by user/item codes already explains most of the ranking signal, and the additional graph information does not add enough discriminative power in the top-10 regime to compensate.

C. Effect of Social Information

Social information helps at the margins but does not produce a step-change. In Social NCF, social regularization improved NDCG@10 from 0.7379 to 0.7422 and reduced MAE from 0.7811 to 0.7794 over base NCF. In GFP-GB, the `neighbor_mean` and `social_degree` features added social context, but their impact is limited by graph sparsity: only 30.8% of users appear in the social graph, so a large fraction of predictions receive no social signal.

D. Ablation Studies

The clearest ablation is between GFP-GB and the untuned LightGBM baseline: the only difference is the addition of 10 graph-propagated features alongside the user/item codes. The resulting RMSE difference is marginal ($\Delta = 0.0018$), while MAE improves by 0.0023. This suggests that graph propagation adds modest value over pure entity-ID features on this dataset, and that the bottleneck is data sparsity rather than model expressiveness.

VIII. INDIVIDUAL REFLECTIONS

A. Cale Smith

I was not expecting to get destroyed by an untuned LightGBM. Damn. I think the biggest reason is that this is a super sparse dataset, even whith the friendship graph, and LightGBM is great at cold-start predictions. Because of the sparsity of the dataset I was relying on global mean for a ton of my predictions and the social regularization that I used just ran into the same problem with an extra step. Instead of directly predicting mean it went through a bunch of extra steps that, if we're being honest, basically ended up at the same mean. We manipulated the embeddings so that friends were in similar vector spaces but, because the graph is so sparse, we still ended up with predictions that were still heavily reflective of the global mean.

The implementation was super fun and I had a blast training it, though it felt like there wasn't enough time to do everything I wanted within the project start/end window. In hindsight I think two things stand out. First, I'm going to use the simplest model I can to start every future ML project, I really suspect that it's an underated strategy with school projects. Second, I tried to add a ton of regularization and to get a double descent situation but it just didn't work. I think this was because, while the dataset was massive I didn't have enough complexity in my model (had to keep the layers and embeddings pretty small so that it would run on my potato-ass laptop), and the sparsity meant that there just wasn't much signal there in the first place.

All in all, sparsity sucks. The end.

B. Advait Shinde

My first attempt was LightGCN [5], which does end-to-end graph convolution over the bipartite user-item graph. It overfit hard — val RMSE bottomed out at 0.9458 by epoch 4 and climbed from there. The dataset is just too sparse for a purely learned approach to generalize.

Pivoting to handcrafted feature propagation (GFP-GB) was the right call for stability. Shrunk statistics propagated through the graph give the model neighborhood context without requiring the graph signal to survive backpropagation. I also tried a GFP-MLP variant (val RMSE 0.9524 in the notebook) before landing on LightGBM as the predictor, since tree models handle the mixed categorical and continuous feature space better.

The honest result: GFP-GB edged the baseline on MAE but not RMSE or ranking. The propagated features do help, but the bottleneck is that 70% of users have no social connections and

TABLE VI
PERFORMANCE COMPARISON ACROSS ALL MODELS (LIBRARYTHING)

Model	Val RMSE	Test RMSE	Test MAE	P@10	R@10	NDCG@10
Global Mean	0.9956	0.9945	0.7852	0.2995	0.7237	0.7248
LightGBM	0.9551	0.9690	0.7510	0.3115	0.7295	0.7491
Tuned LightGBM	0.9480	0.9632	0.7434	0.3140	0.7315	0.7539
NCF	0.9884	0.9922	0.7811	0.3045	0.7270	0.7379
Social NCF	0.9916	0.9919	0.7794	0.3074	0.7283	0.7422
GFP-GB	0.9594	0.9708	0.7487	0.2938	0.7327	0.7209

many items have very few raters, so the neighborhood averages collapse toward the global mean anyway. On a denser graph the approach would likely shine more.

C. Jasper Morgal

To be honest, I am somewhat disappointed in the feature importance of my model. I had high hopes for the social item features, and thought they might be able to benefit the LightGBM a lot. What ended up happening was the model worked best only using a few narrow user and item aggregate features, not dissimilar to the baseline LightGBM.

Through this project, I learned that training recommender systems is very hard. There's a million good sounding ideas and only very few of them actually work. While this model's behavior was not as satisfying as my song reccomender system for the Kaggle challenge, I still had fun optimizing the RMSE.

IX. CONCLUSION

Across all models, the clearest result is that simple, highly optimized tabular methods worked the best on this dataset. The hyper-tuned LightGBM model achieved the best overall performance, with a test RMSE of 0.9632 and the highest NDCG@10 score of 0.7539. Most of this gain came from carefully engineered user/item aggregate features rather than from complex social graph features or neural modeling. The untuned LightGBM baseline was already very competitive, which suggests that on an extremely sparse recommendation dataset, strong entity-level bias features can explain much of the available signal.

The social graph still provided useful but limited information. Social NCF improved over base NCF on ranking metrics, and GFP-GB showed that propagated graph statistics can slightly improve typical error, but neither approach surpassed the LightGBM variants. Overall, our results suggest that social-aware methods are promising but highly dependent on graph coverage and interaction density. For LibraryThing, where most users have few ratings and many users are outside the social graph, the best strategy was to use simple models with careful feature engineering.

REFERENCES

[1] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th International Conference on World Wide Web*, 2017, pp. 173–182.

[4] H. Ma, D. Zhou, C. Liu, M. R. Lyu, and I. King, "Recommender systems with social regularization," in *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*, 2011, pp. 287–296.

[5] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 639–648.